Backdoor Attack Against Split Neural Network-Based Vertical Federated Learning

Ying He, Zhili Shen, Jingyu Hua, *Member, IEEE*, Qixuan Dong, Jiacheng Niu, Wei Tong, *Member, IEEE*, Xu Huang, Chen Li, and Sheng Zhong, *Senior Member, IEEE*

***Abstract*— *Vertical federated learning* (VFL) is being used more and more widely in industry. One of its most common application scenarios is a two-party setting: a participant (i.e., the host), who exclusively owns the labels but possesses insufficient number of features, wants to improve its model performance by combining features from another participant (i.e., the client) of a different business group. The best deep ML architecture suits for this scenario is considered to be *Split Neural Network* (SplitNN), in which each participant runs a self-defined bottom model to learn the hidden representations (i.e., the local embeddings) of its local data and then forwards them to the host, who runs a top model to aggregate both the local embeddings to produce the final predicts. In this paper, we assume the client is malicious and demonstrate that she/he could inject a stealthy backdoor into the top model during the training to misclassify any sample to a pre-selected target class with a high probability by just replacing its local embedding with a special trigger vector regardless of the host-side embedding. This task is non-trivial because existing data poison attacks for backdoor injection in traditional models usually require to modify the labels of a set of trigger-tagged samples of non-target classes, which is impossible here as the client has no rights to access or modify the labels exclusively owned by the host. Targeting this challenge, we propose a SplitNN-dedicated data poison attack which does not require to modify any labels but just replaces the local embeddings of a very small number of target-class samples with a carefully constructed trigger vector during training. The experiments on four datasets show that our attack can achieve an attack rate as high as 94%, while bringing negligible side-effects to the model accuracy. Moreover, it is stealthy enough to resist various anomaly detection methods.**

***Index Terms*— Backdoor attack, split neural network, vertical federated learning.**

Manuscript received 30 March 2023; revised 27 July 2023 and 11 September 2023; accepted 3 October 2023. Date of publication 26 October 2023; date of current version 22 November 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 61872176, Grant 61872179, Grant 61972195, Grant 62002159, and Grant 62372226; in part by the Jiangsu Shuangchuang Talent Program under Grant JSSCBS20210035; and in part by the Leading-Edge Technology Program of Natural Science Foundation of Jiangsu Province, China, under Grant BK20202001. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Edgar Weippl. *(Corresponding authors: Jingyu Hua; Wei Tong.)*

Ying He, Zhili Shen, Qixuan Dong, and Jiacheng Niu are with the Computer Science and Technology Department, Nanjing University, Nanjing 210023, China (e-mail: guapi7878@gmail.com; zhilishen@163.com; 181860016@smail.nju.edu.cn; lingdangsmoke@gmail.com).

Jingyu Hua, Wei Tong, and Sheng Zhong are with the State Key Laboratory for Novel Software Technology and the Computer Science and Technology Department, Nanjing University, Nanjing 210023, China (e-mail: huajingyu@nju.edu.cn; weitong@outlook.com; zhongsheng@nju.edu.cn).

Xu Huang and Chen Li are with Meituan Company, Beijing 100102, China (e-mail: huangxu17@meituan.com; lichen11@meituan.com).

Digital Object Identifier 10.1109/TIFS.2023.3327853

I. Introduction

NOWADAYS, more and more internet companies train various machine learning (ML) models based on their user data to intelligentize their businesses. Nevertheless, a lot of times, an individual entity may possess insufficient data for building a high-quality model, in which cases, it may hope to improve the model's performance by incorporating more valuable data from other independent entities. Unfortunately, privacy-regulations such as GDPR prevent a company from directly disclosing its user data to any other one, which makes such cooperation extremely hard, if not impossible.

Federated learning (FL) [1], one kind of privacy-preserving distributed ML technique proposed by Google recently, is considered as a promising alternative to address this issue. It allows participants to collaboratively train a ML-model by periodically exchanging intermediate computation results rather than their raw data. Since it can make a good balance between the model quality and the user privacy, many top IT companies have devoted to developing various FL platforms.

One typical application scenario of FL is that the data is vertically partitioned among a few participants (usually two), which is also known as vertical FL (VFL) [2]. In other words, the datasets of the participants share the same sample space but differ in the feature space. In the most widely used two-party setting, which is also focused by this paper, there is usually a participant called *the host*, who exclusively owns the label but possesses insufficient number of features, desiring to improve its model performance by incorporating more features from another participant that is called *the client* and might be from a different business group. For instance, a bank with enough number of labels but limited valuable user features for credit risk assessment may seek help from an online shopping company who owns daily shopping features of the users.

Recently, Split learning (SL) has been accepted as a promising technique to implement the above VFL by many leading IT companies (e.g., ByteDance [3] and Tencent [4]) for the two-party setting. It splits the whole neural network into two bottom models and one top model at a middle layer, referred to as the cut layer. Each participant runs a self-defined bottom model, which actually serves as an embedding network to learn the hidden representations (a d-dimension vector, called the local embedding) of the local data partitions. The top model defined and run by the host aggregates the both local embeddings to generate the final output. In the backward propagation, the host will share the gradients of the cut layer

with the client to enable her/him to update the parameters of the bottom model. This architecture is also known as *the split neural network* (SplitNN) [5]. Its advantage is that the participants can freely define their local models without relying on each other, which is extremely desired in a fully distributed learning scenario. Moreover, the client only reveals the highly abstracted embeddings rather than the raw data, which can preserve the user privacy to a great extent.

Nevertheless, as the client is usually from a different business domain, the host cannot guarantee that she/he is always honest and never behaves maliciously during the collaborative training and prediction. In fact, some existing works [6], [7], [8] have shown that an adversarial client may leverage the learned gradients or her/his own local embeddings to steal the labels of both training and predicting samples, which is obviously not expected to occur.

In this paper, we try to demonstrate the existence of another risk: **the first backdoor attack against universal SplitNN-based VFL**, in which, the adversarial client aims to inject a stealthy backdoor in the top model during training so that it will misclassify any sample to a target class with a high probability regardless of the input of the host by just replacing the client-side local embedding with a pre-determined trigger vector. The word 'universal' means this attack is universal to varied models of the host.

Note that it is infeasible for the client here to trivially reuse the existing backdoor injection techniques against traditional centralized or horizontal FL (HFL) models. First of all, the top model is fully a black box to the client, who, thereby, has no ways to directly subvert its neuron parameters to inject the backdoor. The only remaining feasible way is to poison the training data. Unfortunately, existing poison strategies [9], [10] usually require to modify the labels of a set of randomly selected non-target-class samples to the target class after attaching them with a pre-determined trigger. Training on this poisoned dataset will certainly build a strong connection between the trigger and the target label. In our scenario, however, the labels are kept and provided by the host only. The client has no rights to access them, let alone modify them. As a result, if we cannot setup a strong enough connection between the trigger vector (i.e., a false client-side local embedding to trigger the top model to misclassify the samples to the target class) and the target class, the host-side embedding may still guide the top model to give the true prediction.

Targeting the above challenge, we propose a dedicated training poison attack which does not require to tamper with any sample's labels. In this attack, we assume the client has obtained a small amount like 500 of labeled training samples of the target class in advance by some offline approaches. We think it is not hard for her/him to get such a small dataset as she/he can directly buy them from the employees of the host company. This attack then uses an efficient method to generate a trigger vector that can let the client perfectly inject the desired backdoor by just using it to replace her/his local embeddings of the samples within this small auxiliary set during training.

The intuition idea behind this method is to construct a statistically normal vector that is far enough from the areas of the non-target classes in the local embedding space. And thus, for most of the samples in non-target classes, once we replace their client's local embeddings with this vector, they will be highly probably much closer to the poisoned training samples assigned the target class than to the clean samples in their original classes even in the whole embedding space combining the another half embeddings of the host. Satisfying this condition, we think the top model would be easily taught to misclassify them even if the client just poisoned a very small number of target-class training samples. Certainly, to guarantee the stealthiness, our method should also confine both the length and the element values of the generated trigger vector within the normal distributions of natural local embeddings.

In summary, we make the following contribution in this paper:

(1) We propose an effective backdoor attack against the SplitNN-based VFL. In this attack, the adversarial client uses a carefully constructed trigger vector to replace her/his local embeddings of a very small number of pre-fetched target-class samples during training. No labels of any samples need to be tampered with. By doing so, she/he would inject a backdoor into the top model that for a sample even not belonging to the target class, once its client local embedding is replaced with the trigger vector in the prediction, it will be misclassified to the target class with a high probability regardless of its host-side features. We also take measures to increase the stealthiness of the vector, e.g., using a perturbation strategy to increase its variety.

(2) We evaluate this attacks on four heterogeneous datasets including Epsilon [11], Criteo [12], CovType [13] and CIFAR10 [14]. The results show that the averaged attack success rate can reach 88%, 90%, 76% and 88%, respectively, while bringing negligible side-effects to the classification accuracy of the models on normal data.

(3) We consider three possible detection methods for the host: The first one tries to statistically filter out abnormal local embeddings from the client. The second one tries to reverse a potential trigger vector for each class by using the white-box top model. The top model is finally considered to be trojaned if there is one reversed trigger vector being obviously abnormal compared with others. The third one is the confusional autoencoder defense proposed by Liu et al. [7]. Our experiments on four datasets show that neither of them could detect the proposed attack with high rates, which demonstrates its good enough stealthiness.

(4) We evaluate the effectiveness of our attack in multi-participant VFL. We conduct experiments on both binary and multiple classification datasets and our attack still presents a significant security risk to multi-participant VFL that cannot be ignored.

II. Background

In this section, we first introduce the principle of Split Neural Network (SplitNN)– one of the most popular VFL architectures in industry. Then, from the perspective of traditional backdoor attacks, we explain why it is particularly difficult to inject backdoors into SplitNN. Finally, we briefly model the threats.

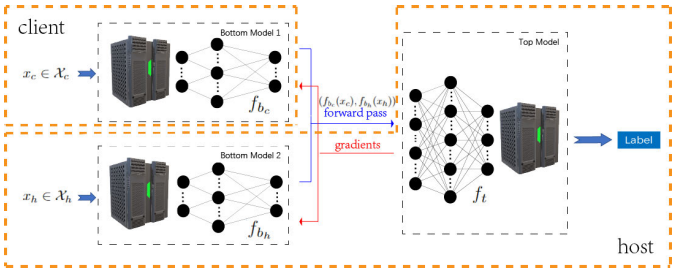


Fig. 1. The architecture of SplitNN-based VFL.

A. SplitNN-Based VFL

VFL [2] tries to promote collaborative machine learning among entities with vertically partitioned data. In other words, the participants' datasets share the sample space but differ in the feature space. Split learning [5] is considered to be one of the most promising technical routes to implement VFL on neural network models, and attracts more and more attentions from the industry, recently.

Fig.1 shows the architecture of SplitNN-based VFL in a two-party setting. In this scenario, there is one participant, referred to as Host h, holding both the labels and a few data features $\mathcal{X}_h$. The remaining features $\mathcal{X}_c$ of the data samples are owned by another participant, referred to as Client c. The whole neural network is split into a top model and two bottom models at a specific middle layer, called the cut layer. Each participant $i \in \{h, c\}$ runs a bottom model $f_{b_i} : \mathcal{X}_i \to \mathbb{R}^d$, which actually serves as a local embedding network to learn hidden representations (a d-dimension vector) of her/his local data. We call these vectors local embeddings below. Note that the architectures of such bottom models are self-determined by the participants and not necessary to be identical. Besides, the host runs the top model $f_t : (\mathbb{R}^d, \mathbb{R}^d) \to \mathcal{Y}$, which aggregates the local outputs from the both bottom models and computes the final outputs. During the backward propagation, the host will share the gradients of the cut layer with the client, who can then further update her/his bottom model f_{b_c}. In the prediction state, the label of an input $x : (x_h, x_c) \in \mathcal{X}$ is also collaboratively predicted by the host and the client: $f_t(f_{b_h}(x_h), f_{b_c}(x_c))$ in formal.

Although the above architecture could be easily extended to the scenarios with more than two participants, VFL is more likely performed between two parties in industry now due to the trust and efficiency issues. Most existing works [6], [8], [15], [16], [17] on SplitNN-based VFL focus on the two-party setting. So, we also only consider the two-party scenarios in this paper.

B. Goals & Challenges of Backdoor Attacks Against SplitNN

As mentioned in [8], SplitNN-based VFL suits best for the scenarios that one participant (i.e., the host), who owns the label but possess limited features, hopes to improve her/his model performance by collaboratively including more features from another participant (i.e., the client). Since the client is usually from an independent business group, it is infeasible for the host to guarantee that she/he is honest. Actually, many existing works [6], [7], [8] have demonstrate that the client may steal the labels of samples via participating in the training.

In this paper, we consider another kind of adversarial client, who aims to inject a stealthy backdoor in the top model during training so that she/he can independently control the predict of any sample with her/his own input partition regardless of the other partition of the host.

More specifically, suppose the train dataset is D_{train}, the adversarial client selects a small subset $D_{Poison} \subset D_{train}$ and poisons the top model by manipulating the local outputs of her/his bottom model f_{b_c} on this set during the training as follows:

$$\hat{f}_{b_c}(x_c) = \begin{cases} \boldsymbol{v}_t, & x \in D_{Poison} \\ f_{b_c}(x_c), & others, \end{cases}$$

where $\boldsymbol{v}_t$ is a pre-determined poison vector, called *trigger vector* below. The goal of this poison attack is to inject a backdoor in the top model which is expected to show the following desired feature at the prediction stage: for any sample $(x, y) \in (\mathcal{X}, \mathcal{Y})$, the predict result of the top model is

$$f_{t_c}(\hat{f}_{b_c}(x_c), f_{b_h}(x_h)) = \begin{cases} y_t, & \hat{f}_{b_c}(x_c) = \boldsymbol{v}_t \\ y, & \hat{f}_{b_c}(x_c) = f_{b_c}(x_c), \end{cases} \quad (1)$$

where y_t is the target label pre-selected by the client. In other words, the top model works correctly under normal conditions. Nevertheless, if the malicious client replaced the local output with the trigger vector, any sample would be assigned the false target label regardless of its real label.

This attack looks quite similar with those traditional backdoors attacks [9], [18], [19], [20], [21], [22], [23] against centralized ML-models or HFL models. Unfortunately, there are at least two particular characteristics of SplitNN making existing mature backdoor injection methods hard be reused in this scenario:

(1) **Label modifiable vs. Label inaccessible**. Training data poison is the most common method to inject a backdoor in traditional centralized or HFL models. In such cases, the adversary is assumed to fully control at least a partial of training sets with all the features and labels. During the training, she/he attaches a pre-determined trigger to some samples under her/his control and directly modifies their original labels to the target one. Training on this poisoned dataset will certainly build a strong connection between the trigger and the target label in the obtained model. Nevertheless, in our scenario present in Fig. 1, the labels of training sets are preserved and provided exclusively by the host. The client is prevented from accessing them, let alone modify them. Therefore, existing data poison attacks are ill-suited here.

(2) **White box vs. Black box**. Besides data poison, direct model modification is another methodology to launch backdoor attacks, especially in the HFL models. However, in this method, the attacker should have the full access to all the parameters of the target model. In other words, it is a kind of white-box attack. However, in our scenario, the top model is a complete black-box to the adversarial client and she/he has no ways to modify it. As a result, this road is blocked, too.

So, the main motivation of this paper is to study whether the malicious client could address these challenges to realize

the goals of the expected backdoor attack in SplitNN. We will present our proposal in Sec.III.

C. Threat Model

As mentioned earlier, this paper focuses on the two-party SplitNN-based VFL described in Sec.II-A. In this model, the label data only belongs to the host. The input data is vertically partitioned into two independent and uncross partials. One partial belongs to the host and the other belongs to the client. They never exchange their raw data at any time.

1) Attacker's Capability: We assume the client is malicious and aims to launch the backdoor attack mentioned in the above subsection. The bottom model f_{b_c} is in her/his full control, which means she/he can arbitrarily tamper with its outputs before uploading them to the host both in the training and the prediction stages. Normally, the clients knows little about any data partitions (including labels) or model parameters kept by the host. The only exception is that we assume the clients has learned the labels of a small number (500 is already enough according to our experiments) of training samples of the target class y_t through some offline approaches before the training. This set is denoted by $DAux_t$. Note that the host-side features of these data are still unknown to the client. We think it is not hard for the adversary to get this small amount of labeled data as she/he can directly buy them from employees of the host.

2) Defender's Capability: Obviously, the host is the defender in this security game. She/he knows little about the bottom model or the raw data partitions of the client. She/he can only rely on checking the received local embeddings from the bottom model of the client to judge whether a backdoor attack is happening.

III. Proposed Backdoor Attacks Against SplitNN-Based VFL

In this section, we will present our approach to implement the backdoor attacks described in Sec.II-B.

A. Overview

Note that our backdoor goal is to achieve Eq. 1: for any sample x, when the client replaces her/his local output (i.e., $f_{b_c}(x_c)$) with a trigger vector $\boldsymbol{v}_t$, it will be misclassified to a pre-determined label y_t by the top model regardless of its original label. Otherwise, it will be correctly classified.

As the top model (f_t) is a complete black box to the client, it is impossible for she/he to directly modify this model to inject the desired backdoor. So, the only remained way is data poison. After all, the half of the inputs to f_t is just the local output of the client's bottom model f_{b_c}, which is in full control of the client and can be arbitrarily manipulated during training.

The major challenge here is all the labels of the training samples are controlled by the host and the clients have no ways to modify. Thus, she/he cannot reuse the existing data poison attacks to directly modify the labels of randomly selected poisoned samples (of which the local outputs have been replaced with $\boldsymbol{v}_t$) to the target label during training to teach the top model to build the connection between $\boldsymbol{v}_t$ and y_t.

So, the problem now becomes how the client can build the above connection in the top model without modifying any labels of training samples. Obviously, in this case, it is meaningless to poison non-target-class samples since replacing their local outputs with $\boldsymbol{v}_t$ during training would introduce connections between $\boldsymbol{v}_t$ and non-target labels, which plays side-effects in the backdoor injection. Thereby, the only remaining way for the client is trying to build a strong connection between $\boldsymbol{v}_t$ and y_t via just poisoning samples of y_t. Recall that we assume the client has obtained $DAux_t$, a small set of training samples of the target label.

This task is extremely challenging because the top model only observes poisoned samples of the target class during training. It is hard to predict what will happen when it observes a non-target-class sample with its client-side local embedding being replaced with $\boldsymbol{v}_t$ in the prediction stage. Note that the input to the top model also includes the embedding from the features owned by the host, which may still guide the top model to give the true prediction. So, finding a proper trigger vector $\boldsymbol{v}_t$ becomes especially critical here. It should satisfy at least the following requirements:

- By just replacing the local embeddings of samples in the small set $DAux_t$ with $\boldsymbol{v}_t$ during training, the client can guarantee the top model would perform as Eq. 1 defines during prediction: if the local output of a sample at the client is replaced with $\boldsymbol{v}_t$, it will be classified to the target label t regardless of its original label. Otherwise, it will be classified correctly.
- $\boldsymbol{v}_t$ should not be statistically abnormal from the local embedding of normal samples of the client. We mainly compare them from three metrics: the abnormality of each element value, the abnormality of the vector length and the outlier degree from the predicted class, which are formally defined in Sec. V-A.

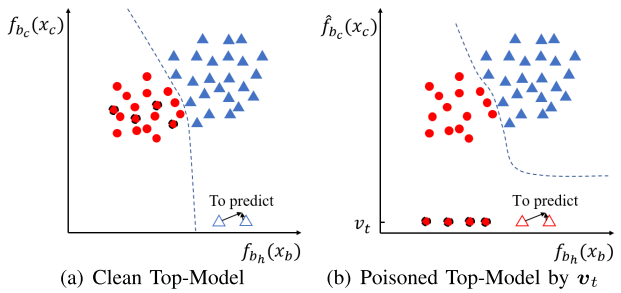
Next, we will first present our method to search for this trigger vector and then introduce some additional strategies to further improve the stealthiness of the attack.

B. Construction of the Trigger Vector

As Sec.II-A describes, the cut layer, i.e., the input vector to the top model is just a combination of the two local embeddings of the bottom models of the client and the host. In that sense, we can simply think that the major task of the top model is learning to partition the whole embedding space, which has been highly abstracted compared with the raw data space.

According to the above analysis, our basic idea is to find a trigger vector $\boldsymbol{v}_t$ that can make those poisoned samples (i.e., with their client-side outputs being replaced with $\boldsymbol{v}_t$) fall in an area as far as possible from those sample areas of non-target-classes regardless of the host-side local embeddings. We think it is much easier for the client to leverage such a trigger vector to mislead the top model to falsely partition the embedding space to meet Eq. 1 even if she/he just poisons the samples in the small set $DAux_t$.

Let us use the simple example in Fig. 2 to illustrate this idea. In this example, we assume the local embeddings of the client



(a) Clean Top-Model (b) Poisoned Top-Model by $\boldsymbol{v}_t$

Fig. 2. A simple example to illustrate our basic idea to search for the trigger vector $\boldsymbol{v}_t$.

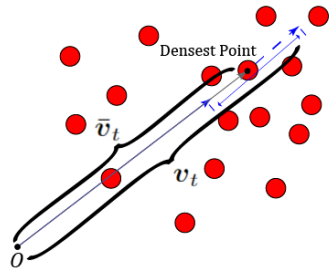


Fig. 3. Illustration of our approach to construct the trigger vector.

and the host are both one-dimensional vectors. As a result, the embedding space is just a 2d-space, where the x-axis and the y-axis represent the host's and the client's embeddings, respectively. There are two data classes, which are denoted by red circles and blue triangles, respectively. We assume the client-side embeddings of the second-class samples are much greater than 0. The left figure shows the possible partition of the embedding space by the top model after a clean training. Comparatively, in the right figure, we assume the client randomly selects four samples of the first class (red circles with dotted black circumferences) and replaces its local embeddings with a value v_t that is close to 0 during the training. So, they must be extremely far away from the area of class two. By doing so, the partition of the embedding space might be changed as the figure shows. In particular, for a sample belonging to the second class, if the client changes its local embedding to v_t, the new top model may classify it to the first class with a high probability just as the red triangles in Fig. 2(b). This is because it is much closer to the four poisoned training samples of the first class than to the normal samples of the second class (blue triangles) even taking into the x-axis value into consideration.

Based on the above idea, we try to find the optimal trigger vector $\tilde{\boldsymbol{v}}_t$ to the following optimization problem:

$$\tilde{\boldsymbol{v}}_t = \max_{\boldsymbol{v}_t \in \mathbb{R}^d} \min_{\substack{(x,y)\in D_{train} \\ y \neq y_t}} \left\| f_{b_c}(x_c) - \boldsymbol{v}_t \right\|_2$$
$$s.t. \quad \begin{cases} \forall_i : \boldsymbol{v_t}[i] \leq \max\limits_{x \in D_t} f_{b_c}(x_c)[i] \\ \forall_{x \in D_t} : \|\boldsymbol{v}_t\|_2 \leq \max\limits_{x \in D_t} \left\| f_{b_c}(x_c) \right\|_2 \end{cases} \tag{2}$$

Here, to guarantee the stealthiness of the attack, we confine both the value of each dimension and the L_2-norm of the whole vector within the normal distributions of the local embeddings of training samples belong to the target class t, i.e., D_t.

Unfortunately, it is straightforward to demonstrate that this problem is NP-hard. What is worse, to find the optimal solution, the client is required to learn a great number of (if not all) labeled samples belonging to each non-target class, which violates our assumption on the client's ability that she/he possesses only $DAux_t$ - a small amount of labeled samples of the target class, and will also significantly increase the attack costs of the client. So, instead of searching for the optimal solution, we try to propose a heuristic method to construct a good enough approximate solution just relying on $DAux_t$.

Intuitively, our final goal is to find a trigger vector $\boldsymbol{v}_t$ which looks normal but is far enough from the client's local embedding of every sample of non-target classes. "far enough" here is unnecessary to mean the farthest. Instead, it only needs to be able to reduce the affects of the host-side local embeddings to some extent that for most samples of non-target classes, once their client-side embeddings are replaced with $\boldsymbol{v}_t$, they will become closer to the poisoned samples of $DAux_t$ than to the normal samples of their original classes in the whole embedding space combining the host's local embeddings just as the example in Fig. 2(b). Based on above, our heuristic method is composed of two steps like Fig. 3:

(1) The client first inputs all the samples in $DAux_t$ into her/his bottom model to obtain their local embeddings. Then she/he selects 50 most relevant embeddings by calculating the Pearson correlation coefficients among them. Next, the client applies the classic mean-shift algorithm on them to find the point $\bar{\boldsymbol{v}}_t$ with the highest density in the local embedding space $\mathbb{R}^d$. We think $\bar{\boldsymbol{v}}_t$ is a potential option for $\boldsymbol{v}_t$ as a dense position where the samples of a class concentrating on is usually not close to the boundary of this class, and thus it is of high probability far from the samples in other classes, which is perfectly desired. Moreover, the principle of the mean-shift algorithm guarantees both the dimensional values and L_2-norm fall in the distributions of normal samples.

(2) To further strength the effect of $\bar{\boldsymbol{v}}_t$, we additionally fine-tune its length by applying a scaling factor α: the final trigger vector $\boldsymbol{v}_t = \alpha \bar{\boldsymbol{v}}_t$. As α increases, the norm of the trigger vector also increases, leading to a stronger impact on the top model and consequently a higher attack success rate. Fig.4 illustrates the impact of α variations on the attack performance. From the figure, we observe that an increase in the number of participants does not alter this relationship. Besides, manipulating the value of α allows for a rapid and significant improvement in the attack success rate. However, this is an ideal scenario. In fact, too large α will cause the trigger vector's length and element values to deviate from those of other normal vectors, making it easily detectable by statistical filtering detection schemes. Its value used in our experiment is $\frac{\max\limits_{x \in D_{train}} \left\| f_{b_c}(x_c) \right\|_2}{\|\bar{\boldsymbol{v}}_t\|_2} \times 0.85$. Based on our practical experience, such a setting is generally a suitable and universal choice.

The above method does not need to know any labeled samples except those in $DAux_t$. According to our experiments,

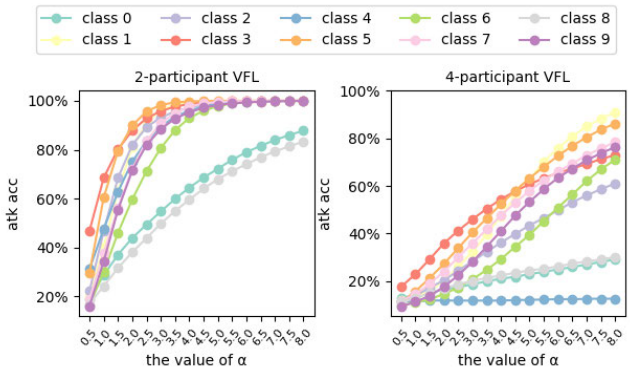


Fig. 4. The impact of α variation on attack success rates across different attack tasks on CIFAR10.

even without poison training, this vector can achieve a high attack rate of 85% to some labels against clean models of some datasets.

C. Poison Training

According to our method of trigger vector construction, we can't intervene the training process from the beginning like common poisoning attacks. This is because the feature map of the cut layer is unreliable at the beginning and the candidate point calculated by the mean-shift algorithm is worth no reference. So, we decide to poison the federated training process at the later stage when the bottom model begins to converge. During poisoning, the client first uses the method in the last subsection to generate a trigger vector and then replaces the local outputs of the those samples in $DAux_t$ before sending them to the host. Besides, since the parameters of the models change all the time, we will periodically update the trigger vector. The update frequency needs to be set to an appropriate value. If the update frequency is too low, it may result in significant changes to the local embeddings of normal samples during the training intervals, resulting in an abnormal trigger vector or model anomalies. Conversely, a too high update frequency may lead to poisoning failure, as each attempt may use a different trigger vector, leading to insufficient training for the model to memorize it.

Alg. 1 shows the whole attack algorithm. The $poison\_epoch$ is the epoch the attacker starts to intervene the training process. The $update\_period$ is the period of updating the trigger vector. We will introduce their values in our experiments in Sec.IV-A. After this process of poison training, the attack rate can be as high as 92% on some datasets.

D. Stealthiness Enhancing of Trigger Vector

By now, the adversarial client can already use the above poison training process to inject the desired backdoor into the top model. However, although we have restricted the element values and the length of the trigger vector, it still face two stealthiness flaws which might be filtered out by the host from the normal local embeddings:

Lack of variety. The client uses the fixed trigger vector to replace the local embeddings of some different samples during the training and prediction. Unfortunately, the original embeddings of these samples are more likely not identical since there are usually slight differences between their raw data even if they belong to the same class.

Lack of Sparsity. In Eq. 2, we confine the value of each dimension of the trigger vector within a normal range. However, this cannot guarantee that the overall distribution of these values is close to the local embeddings. In particular, as we show in Fig. 5(a), in some tasks (e.g., CIFAR-10), the element-wise distribution of a normal local embedding is spare to some extent, i.e., there are a great number of zero dimensions. However, as shown in Fig. 5(b), the trigger vector constructed based on mean-shift algorithm contains few zeros, especially after being amplified in length.

We thus propose an extra perturbation process, which add a random noise to each dimension, to address these two problems. To guarantee the slight sparsity of the final vector, we design different noise distributions for dimensions above and below a threshold like 0.4. We count the average proportion p_0 of zero-dimensions of the natural local embeddings of the samples in $DAux_t$. Then, for those dimensions below 0.4 in the trigger vector, we randomly set a partial of them to 0 to make the total proportion of zero-dimensions reach p_0. For the remaining dimension, we choose to add a small Gaussian noise $\eta \sim N(\mu, \delta)$, where $\mu = 0, \delta \leq 0.5$.

Our experiments in Sec.IV show that such a perturbation scheme has limited effects on the attack success rate. Fig. 5(c) shows that the trigger vector perturbed with this scheme now looks much more similar to the natural local embedding (Fig. 5(a)). We can say that the perturbation scheme successfully helps to enhance the stealthiness of the backdoor attack scheme.

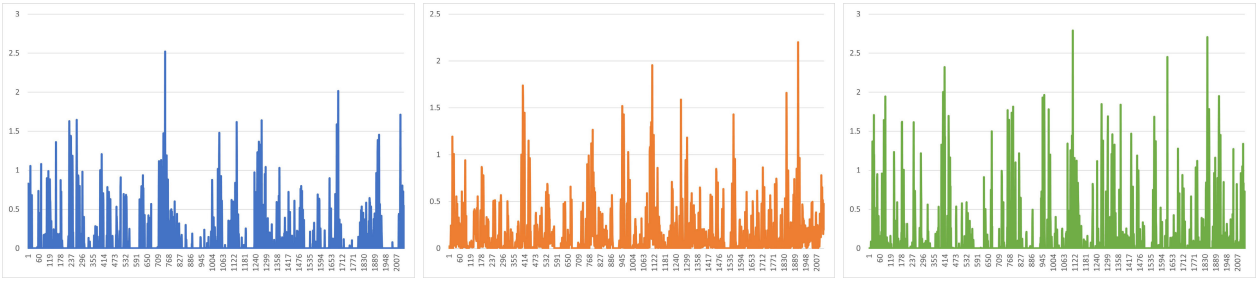
TABLE I
DETAILED CONFIGURATIONS OF HARDWARE & SOFTWARE

Name	Configuration
CPU	both of **two** are *(R) Xeon(R) Silver 4215R CPU @ 3.20GHz* with **8 cores**
GPU	both of **two** are 24GB *GeForce RTX 3090*
Memory	65GB
OS	GNU/Linux(5.4.0-99-generic) 18.04.1-Ubuntu
CUDA	the version is 11.4
GCC	the version is 7.5.0
Python	the version is 3.8.10
PyTorch	the version is 1.7.0

IV. EVALUATION

A. Experiment Setup

Our experiments are conducted on a server, with detailed hardware & software settings listed in Table I. We evaluate the proposed backdoor attack on four popular datasets from different fields: Epsilon [11] – a widely used standard test digital binary classification dataset in the academic community, Criteo [12] – an actual ad-click prediction dataset commonly used to evaluate recommendation systems and SplitNN [6], [8], [16], [17], CovType [13] – a real-world sophisticated structured multiple classification dataset, and CIFAR10 [14] – a well-known image multiple classification dataset. Epsilon and Criteo are used to evaluate the effectiveness of our



(a) Normal Embedding (b) Trigger Vector (c) Perturbed Trigger Vector

Fig. 5. A simple example of the distributions of vectors.

TABLE II
DETAILED INFORMATION OF THE DATASETS

Dataset	Subjects	Number of Labels	Input Size	TrainSet Size
Epsilon	Digital Features	2	50	200000
Criteo	Display Ads	2	$13 + 26$	458000
CovType	Cover Type	7	54	406708
CIFAR10	General objects	10	$32 \times 32 \times 3$	50000

TABLE III
EXPERIMENT PARAMETERS OF DIFFERENT DATASETS

Dataset	Epoch Num	poison epoch	update peroid
Epsilon	100	60	10
Criteo	30	20	-
CovType	100	80	10
CIFAR10	100	80	10

TABLE IV
ATTACK RESULTS OF EPSILON FOR DIFFERENT TARGET CLASSES

Model	Classification Accuracy	Attack Success Rate (without/with noise)	Training Time(s)
clean	77.75%	-	527.89
target_0	77.76%	84.06% / 81.96%	752.30
target_1	77.75%	95.93% / 94.13%	755.09
feature # of the attacker/host: 25/25($DAux_t$ size: 500)			

scheme on common binary tasks in VFL, while CovType is used to demonstrate the scheme's effectiveness in real multi-classification scenarios. Additionally, CIFAR10 is used to verify that our scheme is suitable not only for structured data but also for other data types such as image data. Further details about these datasets are provided in Table II.

The experiment parameters are shown in Table III. Vertical federated models typically have different structures for different tasks. We will introduce the specific data split settings & model structures in the following paragraphs and analyze the evaluation results of the four datasets one by one. Additionally, we test the effects of data splitting and the size of $DAux_t$ on our scheme.

B. Digital Binary Classification – Epsilon

1) Dataset & Data Split Setting: Epsilon is a popular binary classification task with over 400000 objects with 2000 features. To enable faster and more efficient learning, we randomly sample 50 features and half of the records to reduce the dataset size. The 50 continuous features that are similar in form and have no obvious correlation between them. Thus, we mainly focus on the effect of the number of features held by the attacker on the results. We set up four scenarios: the attacker holds 25%/50%/75%/100% of the features, respectively. It is impractical for the attacker to hold no features because that means the host does not need the VFL system.

2) Model Structure: In most scenarios, there are two bottom models, each consisting of 3 layers, with the number of neurons in each layer being proportional to the number of feature inputs. For example, the two bottom models are identical when they have the same number of features. There is also one top model with 2 fully connected layers that is fixed in all experimental settings. The host possesses one bottom model, one top model, and partial features assigned to him/her while the client possesses the other bottom model and remaining features. In the scenario where the attacker holds 100% of the features, there is only one bottom model that belongs to the client, i.e., the attacker.

3) Results & Analysis: We evaluate the effectiveness of our scheme in terms of attack capability and its impact on model accuracy by testing the influence of varying $DAux_t$ sizes under different data segmentation and target classes. We gradually increase the size of the auxiliary set from 100 to 700 to observe the attack's performance and determine a suitable auxiliary set size. Each record in our evaluation results is the average of at least 10 experiments.

Fig.6 illustrates the results. In the first column of Fig.6, we observe that the experimental results generally follow the trend that holding more features and having a larger auxiliary set leads to stronger backdoor attack capability. However, when the $DAux_t$ size is too small, such as 100, the attack effect becomes unstable. When the $DAux_t$ size is larger than 500, our attack performs well. In the second column of Fig.6, we observe that the classification accuracy on the clean dataset of our target models is similar to that of the clean models, indicating that our scheme has a minimal effect on model accuracy.

We focus on the experimental setup of 50% features and 500 auxiliary samples to study more. Because such data segmentation is common and reasonable, and 500 samples are sufficient for the attack. Table IV presents the specific results for this setup, and we find that the attack success rate is high enough. Although the training time takes longer than clean models due to the additional search and trigger vector construction process, the overall training time is still

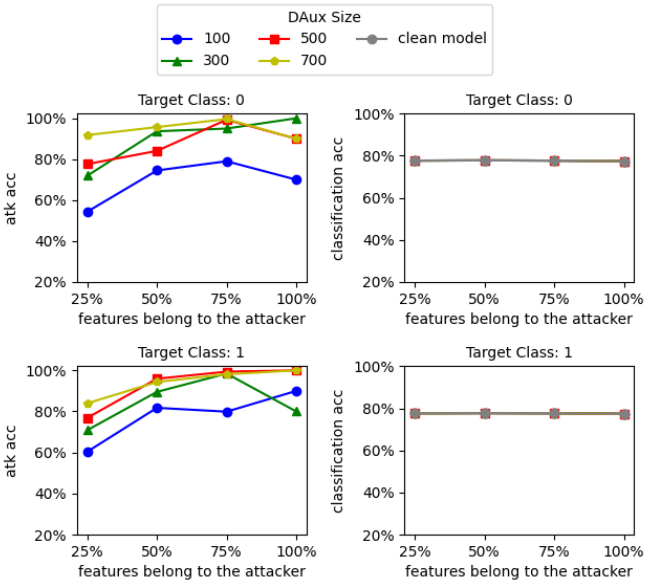


Fig. 6. Influences of different settings on epsilon.

acceptable. In conclusion, our scheme demonstrates excellent performance on Epsilon with only a few auxiliary samples.

C. Ad. Click Prediction – Criteo

1) Dataset & Data Split Setting: Criteo is an online recommendation task that requires predicting whether users will click on a recommendation based on their past 7-day click history. The input includes 13 integer features and 26 categorical features. We also sample each label from the original dataset at the same rate. Besides, we fill the missing integer and categorical values with '0' and '−1' respectively. In the conventional Wide & Deep Learning (WDL) [24] approach, integer features are used in both the wide and deep parts, while categorical features are processed by an embedding layer and only used in the deep part. We follow this approach and propose four data segmentation schemes based on it, where the dimension of the local embeddings for the categorical features is set to 4, and the attacker is assumed to hold 33%/50%/66%/100% of the features. Again, we do not consider the situation where the attacker has no features.

2) Model Structure: We design a SplitNN model based on the mentioned classical prediction model – WDL. In WDL, the wide part is a linear model, while the deep part is similar to a standard DNN model. The final prediction is the mean output of the two networks. For our SplitNN model, we only split the deep component into two bottom models and one top model. Each bottom model has 3 layers, and the top model has 2 fully connected layer. As before, the number of neurons in each layer of bottom models is proportional to the number of feature inputs they have. The top model still remains fixed across all experimental settings. In our most scenarios, the client holds only one bottom model of the deep part, while the host holds the others. In the scenario where the attacker has 100% of the features, there is only one bottom model and one top model in the deep part, and the bottom model belongs to the attacker, who is also the client.

TABLE V
ATTACK RESULTS OF CRITEO FOR DIFFERENT TARGET CLASSES

Model	Classification Accuracy	Attack Success Rate (without/with noise)	Training Time(s)
clean	74.51%	-	422.09
target_0	74.53%	95.38% / 91.20%	528.78
target_1	74.52%	95.47% / 90.30%	524.69
feature # of the attacker/host: 26(sparse features)/13(dense features)($DAux_t$ size: 500)			

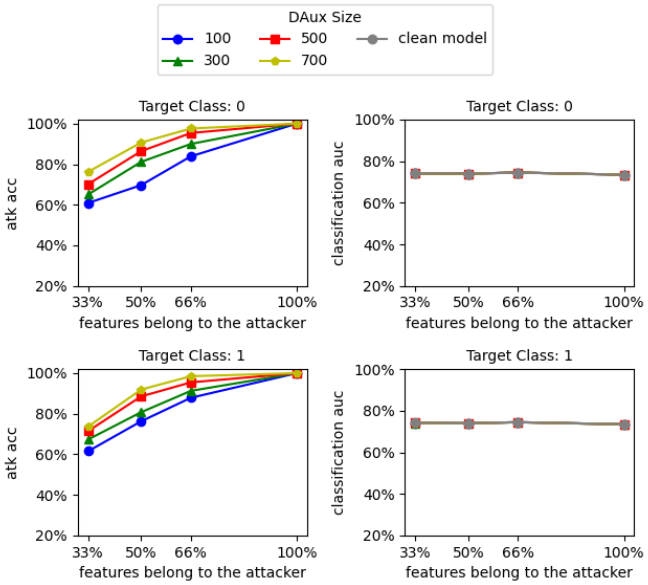


Fig. 7. Influences of different settings on criteo.

3) Results & Analysis: In our evaluation of Criteo, each record represents the average of at least 10 experiments, and the $DAux_t$ size setting remains the same as in our experiments on Epsilon. As shown in Fig.7, the attack success rate varies significantly with the size of the auxiliary set and the number of features held by the attacker. When the attacker has access to 500 auxiliary samples, the attack performance is satisfactory. Therefore, we select the scenario where the attacker holds 66% of the features (i.e., all the categorical features) as the setup for our further studies. This is because the two types of features are quite distinct, and it is more realistic for one party to hold one type of data.

Table V presents the detailed results of our scheme in this setup. It shows that our scheme achieves high attack success rates (at least 90.30%) while having negligible side-effects on the classification accuracy. Additionally, the slightly increased training time compared to that of clean models is acceptable. In general, our proposed scheme performs excellently on the Criteo dataset.

D. Cover Type – CovType

1) Dataset & Data Split Setting: CovType is a large-scale multi-classification dataset related to cover type in real-world scenarios. It has 54 columns of geographic measures as input features and 7 kinds of forest cover as output. We use the built-in dataset of Scikit-Learn [25] directly without any modification. Similar to the previous setup, we evaluate four different attack scenarios where the attacker has access to 25%/50%/75%/100% of the features. Moreover, as CovType

Algorithm 1 Complete Poison Training Process in two-party SplitNN

Input: $DAux_t$, $poison\_epoch$,$update\_period$, learning rate η
Output: Model parameters θ_{b_c}, θ_{b_h}, θ_{t_h}

client and host initialize θ_{b_c}, θ_{b_h}, θ_{t_h}
for i in $range(EPOCH\_NUM)$ **do**
  client:
  $V_c = f_{b_c}(X_c)$
  if $i < poison\_epoch$ **then**
    upload V_c
  else
    if i % $update\_period == 0$ **then**
      $\bar{\boldsymbol{v}}_t$ = mean_shift(V_c)
      $\boldsymbol{v}_t$ = adjust_length($\bar{\boldsymbol{v}}_t$)
    end if
    for $item$ in X **do**
      if $item$ in $DAux_t$ **then**
        $V_c[item] = \boldsymbol{v}_t$
      end if
    end for
    upload V_c
  end if

  host:
  $V_h = f_{b_h}(X_h)$
  $V = concat(V_c, V_h)$
  $\hat{Y} = f_{t_h}(V)$
  $l = loss_func(\hat{Y}, Y)$

  client and host update model parameters:
  ${\theta_{t_h}}^{i+1} = {\theta_{t_h}}^{i} - \eta \frac{\partial l}{\partial \theta_{t_h}}$
  ${\theta_{b_c}}^{i+1} = {\theta_{b_c}}^{i} - \eta \nabla_{\theta_{b_c}} l$
  ${\theta_{b_h}}^{i+1} = {\theta_{b_h}}^{i} - \eta \nabla_{\theta_{b_h}} l$
end for

TABLE VI
ATTACK RESULTS OF COVTYPE FOR DIFFERENT TARGET CLASSES

Model	Classification Accuracy	Attack Success Rate (without/with noise)	Training Time(s)
clean	84.48%	-	640.07
target_0	84.75%	98.32% / 97.73%	812.91
target_1	84.64%	96.01% / 95.07%	816.74
target_2	84.71%	41.17% / 40.42%	806.70
target_3	84.78%	55.06% / 54.84%	824.81
target_4	84.68%	90.54% / 89.30%	816.13
target_5	84.70%	60.61% / 59.61%	815.81
target_6	84.75%	99.98% / 99.73%	816.51
feature # of the attacker/host: 27/27($DAux_t$ size: 500)			

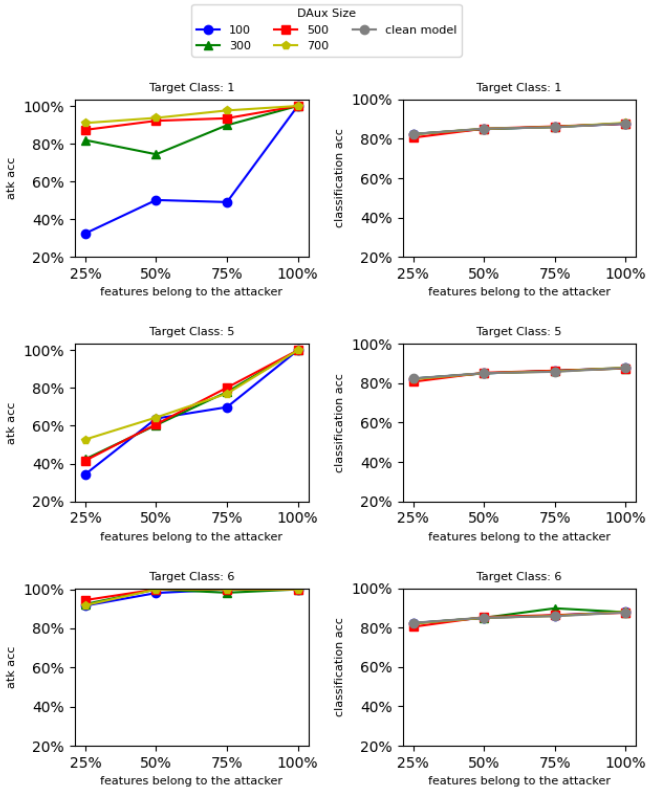


Fig. 8. Influences of different settings on CovType.

is a multiclass dataset with 7 categories, we randomly choose 3 different target classes to show how the attack effect changes with the settings.

2) Model Structure: The model structure we designed for the CovType dataset is similar to that of the Epsilon dataset, with two bottom models and one top model, each consisting of 3 fully connected layers. The number of neurons in each layer of the bottom models is proportional to the number of features they hold. In most scenarios, the host holds one bottom model and one top model, while the client holds the other bottom model. In the scenario where the attacker holds all features, there is only one bottom model and one top model, with the former belonging to the attacker, who acts as the client, and the latter belonging to the host.

3) Results & Analysis: Each record result of CovType in our study is the average of at least 10 experiments. The $DAux_t$ size setting remains the same as before, and Fig.8 presents the results. The first column of Fig.8 shows that the attack success accuracy increases as the attacker has more features and auxiliary samples, which is consistent with the previous binary datasets. However, the attack results are unbalanced across different target class. Our scheme performs similarly on the target $1st$ class as it does on the previous datasets. However, when the target class changes to the $5th$ class, enlarging the auxiliary set hardly improves the attack ability, whereas it becomes extremely easy to attack once the target change to the $6th$ class.

We focus on studying the experimental setup of 50% features and 500 auxiliary samples and find that the situations of the target $2nd$ and $3rd$ classes are similar to that of the $5th$ class. Table VI shows more specific results. We discover that this is due to the unbalanced original class distribution on the cut layer. Classes $2, 3, 5$ are far from other classes on the cut layer of the other bottom model, resulting in the distances of the constructed trigger vector to non-target classes on the cut layer of the attacker's bottom model being shorter than that of the other bottom model. According to our basic idea in Sec.II-B, this affects the attack success rate. However, considering that this is a multi-classification task

TABLE VII
ATTACK RESULTS OF CIFAR10 FOR DIFFERENT TARGET CLASSES

Model	Classification Accuracy	Attack Success Rate (without/with noise)	Training Time(s)
clean	84.63%	-	2349.55
target_0	84.82%	86.46% / 82.85%	2312.52
target_1	84.90%	87.63% / 91.22%	2307.19
target_2	84.74%	86.03% / 87.95%	2353.01
target_3	84.83%	94.53% / 94.65%	2311.38
target_4	84.76%	91.15% / 91.85%	2367.33
target_5	84.74%	91.66% / 92.37%	2308.86
target_6	84.78%	89.49% / 91.40%	2365.97
target_7	84.67%	86.72% / 84.46%	2311.53
target_8	84.88%	71.97% / 76.36%	2311.81
target_9	84.54%	85.50% / 87.02%	2310.34
feature # of the attacker/host: 32*16*32/32*16*32($DAux_t$ size: 500)			

with 7 categories, the minimum attack success rate of 40% suggests that our scheme is still effective to some extent. In fact, we find that the attack success rate is very high for classes that are close to the target class. For instance, when the attacker targets the $2nd$ class, samples originally belonging to the $3rd$ and $5th$ classes can be misled to the target class with a success rate of 93.65%. In conclusion, our attack poses a significant threat to the multi-classification task, while having a minor impact on model accuracy. Additionally, the small increase in training time is acceptable.

E. Image Classification – CIFAR10

1) Dataset & Data Split Setting: CIFAR10 is a classical image classification task where the input is an image with the size of $32\times32\times3$ and the output is the corresponding category. We use the built-in dataset of PyTorch [26] directly without any modification. To carry out the attack, we cut the input image into two parts and distribute them to the two parties. As in the previous setup, we establish four scenarios, where the attacker is given access to 25%/50%/75%/100% of the features, respectively. Similarly to CovType, we also randomly select 3 different target classes to demonstrate how the attack effectiveness varies with different settings.

2) Model Structure: When it comes to image processing, we use different layers on CIFAR10 than the other three datasets. It comprises two bottom models with identical structures consisting of 4 convolution layers and 2 pooling layers, as well as one top model with 3 fully connected layers. These models have fixed structures across all experimental settings, and the output dimension of the bottom models is proportional to the number of feature inputs. Typically, the host has one bottom model and one top model, while the client has another bottom model. However, in the scenario where the attacker holds all features, there is only one bottom model and one top model, owned by the client (i.e., the attacker) and the host, respectively.

3) Results: In our study of CIFAR10, each record result is the average of at least 3 experiments. The $DAux_t$ size setting remains consistent with previous experiments. The results of CIFAR10, as shown in Fig.9, are more regular compared to the other three datasets, and our attack on CIFAR10 has little impact on model accuracy. To maintain consistency with the other datasets, we select the experimental setup of 50% features and 500 auxiliary samples for further experiments.

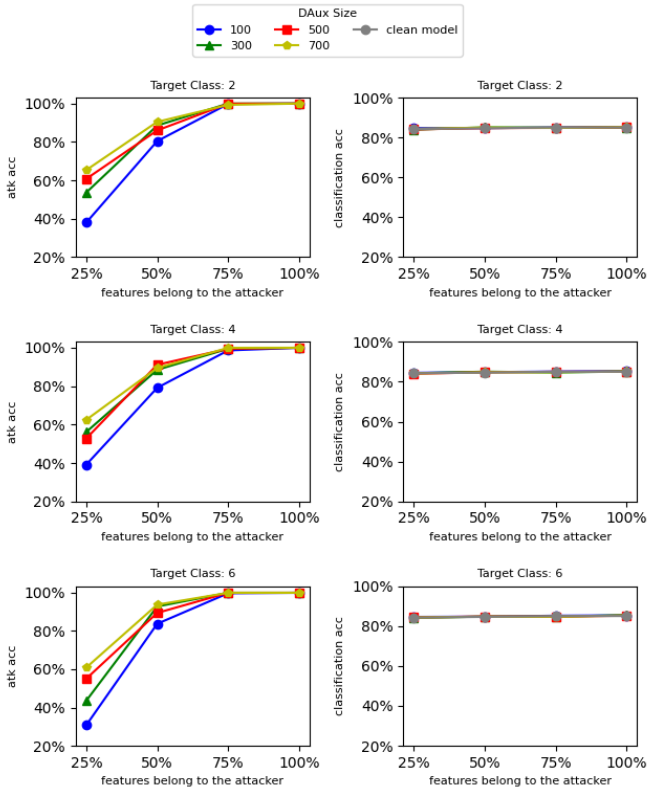


Fig. 9. Influences of different settings on Cifar10.

Table VII displays more detailed results of this setup, where the attack ability is generally satisfactory, except for class 8. The reason behind this is the same as CovType, where the distances between class 8 and other classes tend to be smaller in the cut layer of the attacker's bottom model. As a result, the distances of the constructed trigger vector to non-target classes become shorter, leading to a low attack success rate. Nonetheless, our scheme still exhibits satisfactory performance on CIFAR10, and the training time of our target models is almost the same as that of clean models. These experiments show that our attack works not only on structured data but also on image data.

V. ANOMALY DETECTION AGAINST THE PROPOSED ATTACK

In this section, we consider three possible anomaly detection methods that maybe employed by the host to defense against the proposed backdoor attack. We also test the real performance of these three methods on the previous datasets in our experiments to evaluate the stealthiness of our attack.

A. Statistical Filtering of Abnormal Local Embeddings

The most straightforward defense strategy that the host may use is to filter statistically abnormal local embeddings uploaded by the client. The intuition is that the malicious trigger vectors may deviate from natural outputs of the bottom model in the distributions of the length and the element values. To evaluate the feasibility of this method, we define three

TABLE VIII
STATISTICAL FILTERING RESULTS FOR MODELS ON EPSILON

Target Class		$A_{\boldsymbol{v}_t[i]}$	$A_{\|\boldsymbol{v}_t\|_2}$	$A_{outlier}$	Anomaly Rate
0	T	0/200	0/200	0/200	0.00%
	N	0/200	0/200	0/200	0.00%
1	T	0/200	0/200	0/200	0.00%
	N	2/200	0/200	0/200	1.00%

TABLE IX
STATISTICAL FILTERING RESULTS FOR MODELS ON CRITEO

Target Class		$A_{\boldsymbol{v}_t[i]}$	$A_{\|\boldsymbol{v}_t\|_2}$	$A_{outlier}$	Anomaly Rate
0	T	7/200	0/200	0/200	3.50%
	N	1/200	0/200	0/200	0.50%
1	T	0/200	0/200	0/200	0.00%
	N	0/200	0/200	0/200	0.00%

TABLE X
STATISTICAL FILTERING RESULTS FOR MODELS ON COVTYPE
(T: TRIGGER VECTOR, N: NATURAL EMBEDDING)

Target Class		$A_{\boldsymbol{v}_t[i]}$	$A_{\|\boldsymbol{v}_t\|_2}$	$A_{outlier}$	Anomaly Rate
0	T	0/60	0/60	0/60	0.00%
	N	0/60	0/60	0/60	0.00%
1	T	0/60	0/60	0/60	0.00%
	N	0/60	0/60	0/60	0.00%
2	T	0/60	0/60	0/60	0.00%
	N	0/60	0/60	0/60	0.00%
3	T	0/60	0/60	0/60	0.00%
	N	0/60	0/60	0/60	0.00%
4	T	0/60	0/60	0/60	0.00%
	N	1/60	0/60	0/60	1.66%
5	T	0/60	0/60	0/60	0.00%
	N	0/60	0/60	0/60	0.00%
6	T	0/60	0/60	0/60	0.00%
	N	0/60	0/60	0/60	0.00%

TABLE XI
STATISTICAL FILTERING RESULTS FOR MODELS ON CIFAR10
(T: TRIGGER VECTOR, N: NATURAL EMBEDDING)

Target Class		$A_{\boldsymbol{v}_t[i]}$	$A_{\|\boldsymbol{v}_t\|_2}$	$A_{outlier}$	Anomaly Rate
0	T	0/60	0/60	0/60	0.00%
	N	7/60	0/60	0/60	11.66%
1	T	0/60	0/60	0/60	0.00%
	N	1/60	0/60	0/60	1.66%
2	T	0/60	0/60	0/60	0.00%
	N	4/60	0/60	0/60	6.66%
3	T	0/60	0/60	0/60	0.00%
	N	2/60	0/60	0/60	3.33%
4	T	1/60	0/60	0/60	1.66%
	N	2/60	0/60	0/60	3.33%
5	T	2/60	0/60	0/60	3.33%
	N	5/60	0/60	0/60	8.33%
6	T	0/60	0/60	0/60	0.00%
	N	1/60	0/60	0/60	1.66%
7	T	3/60	0/60	0/60	5.00%
	N	7/60	0/60	0/60	11.66%
8	T	0/60	0/60	0/60	0.00%
	N	4/60	0/60	0/60	6.66%
9	T	0/60	0/60	0/60	0.00%
	N	4/60	0/60	0/60	6.66%

metrics to measure the abnormal degree of a specific trigger vector $\boldsymbol{v}_t$.

(1) The abnormality of each element value $\boldsymbol{v}_t[i]$: $A_{\boldsymbol{v}_t[i]} = \frac{|\boldsymbol{v}_t[i]-\boldsymbol{avg}_t[i]|}{\boldsymbol{avg}_t[i]}$, where $\boldsymbol{avg}_t[i] = \underset{x\in D_t}{Average}(f_{b_c}(x_c)[i])$, and D_t denotes the samples of class t in the training set.

(2) The abnormality of the vector length: $A_{\|\boldsymbol{v}_t\|_2} = \frac{|\|\boldsymbol{v}_t\|_2-avgLen_t|}{avgLen_t}$, where $avgLen_t = \underset{x\in D_t}{Average}(\left\| f_{b_c}(x_c)\right\|_2)$.

(3) The outlier degree from its class: $A_{outlier} = CosineSimilarity(\boldsymbol{v}_t, \boldsymbol{Dst}_t)$, where $\boldsymbol{Dst}_t$ denote the densest point returned by applying the mean-shit algorithm on the local embeddings of samples in D_t.

We set a threshold for each abnormality and so long as any one of them exceeds the corresponding threshold, we consider the local embedding being examined is abnormal. In our experiment, each threshold is simply set to the maximum abnormality of those natural local embeddings of the samples in the 10 clean models.

We apply the above approach to check 20 trigger vectors and 20 normal embeddings, which are both randomly selected, for each model with a specific target class. Specially, we select 10/10/3/3 backdoored models for Epsilon/Criteo/CovType/CIFAR10, respectively. 'Anomaly Rate' refers to the percentage of abnormal embeddings in the corresponding detected embeddings. The results on the four datasets are shown in Table VIII, IX, X and XI. The first line ('T') and the second line ('N') for each target class show the filtering results of perturbed trigger vectors and natural client embeddings, respectively. We find that the highest anomaly rate of all the testing trigger vectors is 5.00% from CIFAR10, indicating that the trigger vectors generated by the attacker are relatively difficult to detect by a statistical filtering approach. Furthermore, the corresponding natural embeddings, which are not supposed to contain any anomalies, have a false positive rate of 11.66%. The results suggest that the statistical filtering approach cannot effectively defend against the proposed attack.

B. Reverse Engineering-Based Detection

Besides statistically filtering the uploaded local embeddings, we consider another detection measure by directly checking the anomaly of the top model as it is a white-box to the host. Inspired by the SOTA backdoor detection scheme – Neural Cleanse [27], this method tries to reverse a potential trigger vector for each class.

Specifically, for every class, it uses the top model to search for a vector as small as possible that once the host directly takes it as the client-side embedding, all the samples will be misclassified to this class regardless of the server-side embeddings. Then, we compare these reversed trigger vectors for different classes, and the top model is considered backdoored if any of them is obviously abnormal. Here, we define two metrics A_{euc} and A_{norm} to measure the anomaly degree of a top model f_t based on the set V_{rev} of reserved trigger vectors. For multiple classification datasets, we use the same technique as Neural Cleanse to calculate the metrics, which is based on MAD [28]. Since it does not work for binary classification datasets, we calculate them as followings:

(1) $A_{euc}(f_t) = \frac{euc_{max}-euc_{min}}{euc_{max}}$, where $euc_{max} = \max_{\boldsymbol{v}_t\in V_{rev}} \|\boldsymbol{v}_t - \boldsymbol{Dst}_t\|_2$.

(2) $A_{norm}(f_t) = \frac{norm_{max}-norm_{min}}{norm_{max}}$, where $norm_{max} = \max_{\boldsymbol{v}_t\in V_{rev}} \|\boldsymbol{v}_t\|_2$.

The intuition of this method is that our attack introduces a trigger vector with the restricted length and values to poison the client inputs of some target-class samples during training, which creates a shortcut from other classes to the target one within the top model. As a result, there might be observable

TABLE XII
REVERSE ENGINEERING-BASED DETECTION RESULTS ON EPSILON

Anomaly Metrics	Reference Range	Average Value		# of detected	Overall DetectRate
		0	1		
A_{euc}	$0 \sim 0.62$	0.35	0.24	2/20	2/20
A_{norm}	$0 \sim 0.83$	0.59	0.42	1/20	

TABLE XIII
REVERSE ENGINEERING-BASED DETECTION RESULTS ON CRITEO

Anomaly Metrics	Reference Range	Average Value		# of detected	Overall DetectRate
		0	1		
A_{euc}	$0 \sim 0.21$	0.11	0.10	2/20	2/20
A_{norm}	$0 \sim 0.33$	0.11	0.09	1/20	

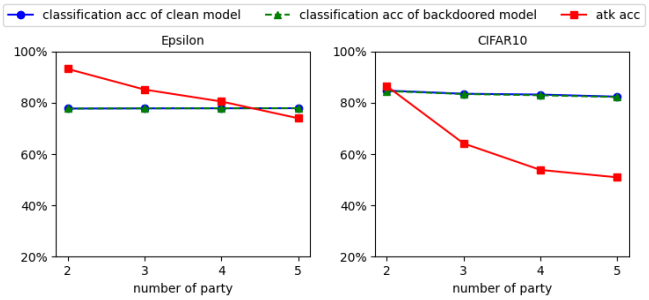


Fig. 10. Influences of different party number.

differences between the reversed trigger vector for the target class and those for other classes.

We trained 10 malicious models for each target class of Epsilon and Criteo, and 3 for each target class of CovType and CIFAR10. The detection results are shown in Table XII,XIII,XIV and XV, respectively. The detection rates are all less than or equal to 13.33%, indicating that the detection of the malicious models is difficult. Particularly, in CovType, the detection rate is as low as 4.76%. The results demonstrate that the proposed attack has high stealthiness.

C. Confusional Autoencoder Label Protection

Liu et al. [7] propose a label privacy protection scheme named CAE (i.e. confusional autoencoder) and claims that it could make backdoor attacks in VFL more difficult. We also test the robustness of our attack using CAE. CAE involves training a pair of encoder and decoder to map each original label to a fake label. The one-hot encoding of the fake label is designed to have minimal differences in classification probabilities between different classes. For example, if the original label is 'cat', the sample label may have a one-hot encoding of $[1, 0]$, whereas the corresponding fake label could be $[0.49, 0.51]$. We replicate the scheme and take experiments on Epsilon, CovType and CIFAR10. After modifying the WDL model to adapt to CAE defense, the primary task accuracy experienced a significant decrease from 74% to 67% on Criteo. Hence we decide not to conduct experiments on it.

Table XVI,XVII,XVIII show the attack performances of backdoored models with the protection of CAE. Firstly, we observe that across all datasets and various attack settings, the primary task accuracy of models slightly decreases after implementing CAE protection, although the differences are not significant. On Epsilon, our attack is hardly affected and even demonstrated higher performance. On CIFAR10, CAE has a minimal impact on our attack, except for a significant decrease in attack success rate for the target class 4, which dropped by approximately 6%. However, the attack effectiveness for other classes remains largely unchanged. Considering that the attack rate for the target class 2 also increased by 5%, overall, we believe that CAE does not hinder our attack on CIFAR10. The performance changes on CovType. Except for the target class 0 and 1, the attack success rate of other target classes decreases significantly. Upon in-depth analysis, we find that there is a severe class imbalance among CovType, where classes 2 to 6 accounting for less than 6% each. Besides, we observe that the addition of CAE significantly reduced the classification accuracy for certain classes in most poorly-performing models, such as class 4, which drops from 44% to around 11%. We speculate that this decline in class recognition ability has an impact on the effectiveness of our attack. However, overall, CAE could not prevent our attack on CovType, as the attack success rate remains high at 84% when the target is class 0 or 1.

In conclusion, the effectiveness of CAE against our attack is not satisfactory. This result is reasonable since our attack does not rely on the gradients backward from the top model or infer labels from the gradients. Therefore, we are not the target of defense for CAE.

VI. ANALYSIS OF EXPANDING TO MULTI-PARTICIPANT VFL

We conduct a preliminary investigation into the feasibility of our attack scheme in multi-participant VFL. We gradually increase the number of participants from 2 to 5, assuming that different participants hold the same number of features, while label data solely belongs to the host. Our experiments are conducted using one binary classification dataset (i.e. Epsilon) and one multi-classification dataset (i.e. CIFAR10). The target label is fixed to '0'. Each record represents the average of at least 3 repetitions of experiments.

From Fig.10, we find that the classification accuracy remains stable, regardless of the number of participants or whether backdoors are injected. As the number of participants increases, the success rate of attacks gradually decreases. Considering that the proportion of features held by the attacker decreases, the influence of the data and model held by the attacker on the primary task decreases, making the result reasonable. Besides, when the number of party reaches 5, the attack success rate of Epsilon remains high, exceeding 70%. For CIFAR10, an attack success rate of over 50% also poses a highly threatening risk for a dataset with 10 classes. Certainly, even in multi-participant scenarios, our proposed attack scheme still presents a significant security risk to VFL that cannot be ignored.

Additionally, we examine how the attacker's feature proportion affects attack effectiveness in multi-participant scenarios. We keep the number of participants fixed at 4 and adjust the attacker's feature proportions to 20%, 30%, 40%, and 50%, while evenly distributing the remaining feature dimensions

TABLE XIV
REVERSE ENGINEERING-BASED DETECTION RESULTS ON COVTYPE

Anomaly Metrics	Reference Normal Range	Average Value							# of Detected	Overall DetectRate
		0	1	2	3	4	5	6		
A_{euc}	$0 \sim 1.43$	0.81	0.90	0.89	0.85	0.93	0.81	0.88	0/21	1/21
A_{norm}	$0 \sim 13.30$	4.42	1.79	4.27	3.80	3.71	4.70	7.24	1/21	

TABLE XV
REVERSE ENGINEERING-BASED DETECTION RESULTS ON CIFAR10

Anomaly Metrics	Reference Normal Range	Average Value										# of Detected	Overall DetectRate
		0	1	2	3	4	5	6	7	8	9		
A_{euc}	$0 \sim 5.52$	2.92	7.71	4.24	2.42	2.52	1.94	2.09	2.89	4.47	2.51	3/30	4/30
A_{norm}	$0 \sim 5.22$	3.01	7.07	4.38	2.34	2.46	2.12	2.20	2.91	4.26	2.47	4/30	

TABLE XVI
RESULT ON EPSILON WITH CAE

Model	Classification Accuracy (without/with CAE)	Attack Success Rate (without/with CAE)	Training Time(s) (without/with CAE)
clean	77.75% / 77.61%	-	527.89 / 662.55
target_0	77.76% / 77.66%	84.06% / 93.76%	752.30 / 975.34
target_1	77.75% / 77.65%	95.93% / 96.46%	755.09 / 965.04
feature # of the attacker/host: 25/25($DAux_t$ size: 500)			

TABLE XVII
RESULT ON COVTYPE WITH CAE

Model	Classification Accuracy (without/with CAE)	Attack Success Rate (without/with CAE)	Training Time(s) (without/with CAE)
clean	84.48% / 83.87%	-	640.07 / 614.31
target_0	84.75% / 84.05%	98.32% / 98.64%	812.91 / 766.39
target_1	84.64% / 84.18%	96.01% / 84.18%	816.74 / 768.36
target_2	84.71% / 84.04%	41.17% / 26.11%	806.70 / 769.93
target_3	84.78% / 84.21%	55.06% / 35.06%	824.81 / 764.28
target_4	84.68% / 84.14%	90.54% / 33.81%	816.13 / 772.27
target_5	84.70% / 84.26%	60.61% / 46.81%	815.81 / 764.33
target_6	84.75% / 84.19%	99.98% / 38.43%	816.51 / 769.51
feature # of the attacker/host: 27/27($DAux_t$ size: 500)			

TABLE XVIII
RESULT ON CIFAR10 WITH CAE

Model	Classification Accuracy (without/with CAE)	Attack Success Rate (without/with CAE)	Training Time(s) (without/with CAE)
clean	84.63% / 83.90%	-	2349.55 / 2303.62
target_0	84.82% / 84.18%	86.46% / 82.54%	2312.52 / 2332.67
target_1	84.90% / 84.31%	87.63% / 92.83%	2307.19 / 2334.56
target_2	84.74% / 83.84%	86.03% / 86.58%	2353.01 / 2314.53
target_3	84.83% / 84.02%	94.53% / 90.39%	2311.38 / 2319.83
target_4	84.76% / 84.09%	91.15% / 85.20%	2367.33 / 2305.98
target_5	84.74% / 84.16%	91.66% / 90.13%	2308.86 / 2314.31
target_6	84.78% / 84.13%	89.49% / 89.17%	2365.97 / 2309.75
target_7	84.67% / 84.01%	86.72% / 86.11%	2311.53 / 2314.93
target_8	84.88% / 84.17%	71.97% / 76.50%	2311.81 / 2316.44
target_9	84.54% / 84.13%	85.50% / 84.93%	2310.34 / 2320.72
feature # of the attacker/host: 32*16*32/32*16*32($DAux_t$ size: 500)			

among the other three participants. As shown in Fig.11, consistent with the two-party scenario, we observe that as the attacker's feature proportion increases, the attack success rate also rises.

Finally, we conduct tests on the stealthiness of backdoors in multi-participant VFL. We configure the experiments with 4 participants, each holding an equal 25% share of the features. We train 10 clean models and 10 backdoor models on both Epsilon and CIFAR10. Since we have already demonstrated in our two-party experiments that our backdoor are not the target of CAE, we only test the remaining two detection schemes in this case. Table XIX shows the results of statistical filtering detection scheme. Notably, none of the vector triggers are detected as anomalies. These results closely resemble those

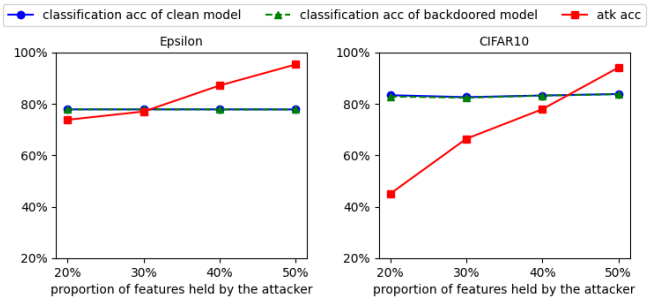


Fig. 11. Influences of the proportion of features held by the attacker on the attack success rate.

TABLE XIX
STATISTICAL FILTERING DETECTION RESULTS IN MULTI-PARTICIPANT VFL

Dataset		$A_{\boldsymbol{v}_t[i]}$	$A_{\|\boldsymbol{v}_t\|_2}$	$A_{outlier}$	Anomaly Rate
Epsilon	T	0/200	0/200	0/200	0.00%
	N	0/200	0/200	0/200	0.00%
CIFAR10	T	0/200	0/200	0/200	0.00%
	N	6/200	0/200	0/200	3.00%

observed in two-participant scenarios. Table XX shows the reverse engineering-based detection results on Epsilon. The abnormality indices of the backdoor models are significantly below the thresholds, and no models are detected. Table XXI shows the detection results on CIFAR10. The average abnormality indices remains below the thresholds, resulting in a detection rate of 20.00%, which is an improvement compared to the 13.33% observed in the two-party scenario but still within an acceptable range. We posit that this phenomenon may be attributed to the reduced influence of the attacker on the top model when multiple parties are involved. Consequently, the success rate of the trigger vector may be lower, leading to an increased impact of poisoning on the backdoor, thereby introducing more anomalies into the model. Nevertheless, our attack scheme maintains good stealthiness in the overall results.

We also observes that the reverse engineering detection scheme becomes more challenging in multi-participant scenarios, requiring more training epochs and higher learning rates. Moreover, when the host does not have a specific target of suspicion, performing detection for each participant incurs greater computational and time overheads.

TABLE XX
REVERSE ENGINEERING-BASED DETECTION RESULTS ON EPSILON IN MULTI-PARTICIPANT VFL

Anomaly Metrics	Reference Range	Average Value	# of detected	Overall DetectRate
A_{euc}	$0 \sim 0.92$	0.45	0/10	0/10
A_{norm}	$0 \sim 0.90$	0.48	0/10	

TABLE XXI
REVERSE ENGINEERING-BASED DETECTION RESULTS ON CIFAR10 IN MULTI-PARTICIPANT VFL

Anomaly Metrics	Reference Range	Average Value	# of detected	Overall DetectRate
A_{euc}	$0 \sim 4.54$	3.57	2/10	2/10
A_{norm}	$0 \sim 4.13$	3.47	2/10	

VII. RELATED WORKS

Risks on SplitNN-based VFL – Existing researches on security risks of SplitNN-based VFL mainly focus label leakage. These works can be divided into two classes by the leakage source: One is inferring labels from the downloaded gradients, the other is inferring labels from the features of the output vectors in the cut layer. In the first class, the shortest distance attack proposed in [6] uses the downloaded gradient g to infer labels of binary classification tasks. If g is more close to the gradient g_0 from the sample to class 0, they think the label of the sample is 0. This is because the gradients tend to tilt in direction in the binary datasets. Liu et al. [7] propose an inference attack on multiple classification VFL models with softmax function layer. According to the inherent mathematical calculation method of softmax function, this method uses the correlation theorem of rank to infer the label information. Although this method has strict mathematical reasoning verification as the guarantee of attack, it is limited to the case that the top network is only a single softmax activation layer. This method fails when the top network architecture is unknown or more full connection layers are added. In the second class, label inference attack [8] is a typical method which uses the semi-supervised learning with a few auxiliary labelled samples to train a model locally to predict the classification of the top model. However, it is limited to stealing labels without deploying a real attack. Compared with these works, our attack considers both binary and multiple classification tasks, and allows the host to design the top model with any structures. At the same time, the attacker knows nothing of the top model.

There are a few contemporaneous works that focus on backdoor attacks in VFL. These works [29], [30] typically assume either known prediction information or train a local top model using an auxiliary set to simulate real prediction information. They optimize the specialized vector generated by the attacker's bottom model through backward propagation by computing the loss function between the predicted values and the target class. LB-RA [29] lacks many constraints on the special vector, resulting in significant differences between it and normal sample embeddings. In contrast, our approach ensures consistency between the uploaded special vector and normal embedding vectors in terms of norm and value distribution. Hijack VFL [30] generates single-source backdoor, while our attack is more universal across all source classes. Moreover, our attack requires less information to be deployed.

Backdoor Attack in DNN – Traditional backdoor attacks usually inject backdoors by data poisoning or model modification [9], [18], [19], [20], [21]. These attacks are most limited to the traditional DNN. There are some backdoor attacks in HFL that poison the local model to affect the global model [22], [23]. Model replacement attack [22] replace the joint model with a malicious backdoor model directly. Through modifying the weights of the local model to influence the federated predictions, the attacker can transfer the specific label to the target label. DBA [23] propose a distributed backdoor attack on HFL. It splits the original global trigger into local patterns and embeds them into different attackers. Such design enhances the robustness and stealthiness of the attack. However, as what we say in Sec.II-B, these attacks can not be implemented in VFL because the labels and the top model structures are unknown.

VIII. DISCUSSION OF POTENTIAL DEFENSES AND FRAMEWORK DESIGNS

Our scheme holds a distinct advantage in terms of both attack effectiveness and stealthiness, posing significant challenges for defenses and the development of secure VFL designs. Defense strategies and VFL frameworks may approach the problem from the perspective of how to disrupt the attack. For example, during the training process, defenders may consider artificially reducing the participants' feature proportions and disrupting potential malicious vectors by randomly discarding different dimensions or filling them with random values in the feature vectors uploaded by participants. However, the challenge lies in striking the right balance between defense capability and maintaining the accuracy of the primary task. Alternatively, defenders could explore methods to further investigate and differentiate top models with backdoors from clean top models. However, the challenge lies in the fact that our attack has a relatively minor impact on models when the attacker holds a large number of feature dimensions or particularly significant dimensions.

Besides, in a multi-participant scenario, the host or VFL framework designers may consider harnessing the collective power of multiple participants by implementing a 'majority rules' strategy. The host can independently predict using embeddings uploaded by the suspicious participant and embeddings from other participants. If there are significant differences between the two results, it can be inferred to some extent that the participant is malicious. However, the challenge lies in the varying importance of different feature components for predicting the primary task, which can lead to false positives when embeddings from less crucial feature dimensions produce inconsistent prediction results.

IX. CONCLUSION

In this paper, we propose an aggressive and stealthy backdoor attack against two-party SplitNN in vertical federated learning. This is the first universal backdoor attack as far as we know, that doesn't limit the model structure. We design

a trigger vector to marginally poison the uploaded natural vectors, and thus plant a backdoor into the top model owned by the host. Besides, we design an appropriate noise to increase the variety of the trigger vector. The experiments shows that our scheme can successfully attack four different popular datasets and keep itself stealthy against various detection approaches at the same time. Besides, we evaluate our attack in multi-participant VFL, and prove that it is a threat to multi-participant VFL as well. We strongly recommend that label owners actively seek collaboration with multiple participants and avoid over-reliance on data provided by any single participant.

REFERENCES

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," 2016, *arXiv:1602.05629*.

[2] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," 2019, *arXiv:1902.04885*.

[3] *ByteDance*. Accessed: Mar. 29, 2023. [Online]. Available: https://www.bytedance.com/zh

[4] *Tencent*. Accessed: Mar. 29, 2023. [Online]. Available: https://www.tencent.com/zh-cn/index.html

[5] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," 2018, *arXiv:1812.00564*.

[6] X. Yang, J. Sun, Y. Yao, J. Xie, and C. Wang, "Differentially private label protection in split learning," 2022, *arXiv:2203.02073*.

[7] T. Zou et al., "Defending batch-level label inference and replacement attacks in vertical federated learning," *IEEE Trans. Big Data*, early access, Jul. 19, 2022, doi: 10.1109/TBDATA.2022.3192121.

[8] C. Fu et al., "Label inference attacks against vertical federated learning," in *Proc. 31st USENIX Secur. Symp. (USENIX Security)*, 2022, pp. 1397–1414.

[9] T. Gu, B. Dolan-Gavitt, and S. Garg, "BadNets: Identifying vulnerabilities in the machine learning model supply chain," 2017, *arXiv:1708.06733*.

[10] J. D. Tygar, "Adversarial machine learning," in *Proc. AISec*, 2011, pp. 43–58.

[11] *Epsilon Dataset*. Accessed: Mar. 29, 2023. [Online]. Available: https://catboost.ai/en/docs/concepts/python-reference_datasets_epsilon

[12] *Criteo Dataset*. Accessed: Mar. 29, 2023. [Online]. Available: https://www.kaggle.com/c/criteo-display-ad-challenge/data

[13] *Covertype Dataset*. Accessed: Mar. 29, 2023. [Online]. Available: https://archive.ics.uci.edu/ml/datasets/covertype

[14] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," M.S. thesis, Univ. Toronto, Toronto, ON, Canada, 2009.

[15] D. Pasquini, G. Ateniese, and M. Bernaschi, "Unleashing the tiger: Inference attacks on split learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 2113–2129.

[16] O. Li et al., "Label leakage and protection in two-party split learning," 2021, *arXiv:2102.08504*.

[17] J. Sun, X. Yang, Y. Yao, and C. Wang, "Label leakage and protection from forward embedding in vertical federated learning," 2022, *arXiv:2203.01451*.

[18] X. Chen, C. Liu, B. Li, K. Lu, and D. Song, "Targeted backdoor attacks on deep learning systems using data poisoning," 2017, *arXiv:1712.05526*.

[19] Y. Liu et al., "Trojaning attack on neural networks," in *Proc. 25th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2018, pp. 1–16.

[20] J. Lin, L. Xu, Y. Liu, and X. Zhang, "Composite backdoor attack for deep neural network by mixing existing benign features," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2020, pp. 113–131.

[21] Y. Liu, X. Ma, J. Bailey, and F. Lu, "Reflection backdoor: A natural backdoor attack on deep neural networks," 2020, *arXiv:2007.02343*.

[22] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," 2018, *arXiv:1807.00459*.

[23] C. Xie, K. Huang, P.-Y. Chen, and B. Li, "DBA: Distributed backdoor attacks against federated learning," in *Proc. Int. Conf. Learn. Represent.*, 2020, pp. 1–19.

[24] H.-T. Cheng et al., "Wide & deep learning for recommender systems," 2016, *arXiv:1606.07792*.

[25] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Nov. 2011.

[26] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–12.

[27] B. Wang et al., "Neural cleanse: Identifying and mitigating backdoor attacks in neural networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 707–723.

[28] F. R. Hampel, "The influence curve and its role in robust estimation," *J. Amer. Stat. Assoc.*, vol. 69, no. 346, pp. 383–393, Jun. 1974.

[29] Y. Gu and Y. Bai, "LR-BA: Backdoor attack against vertical federated learning using local latent representations," *Comput. Secur.*, vol. 129, Jun. 2023, Art. no. 103193.

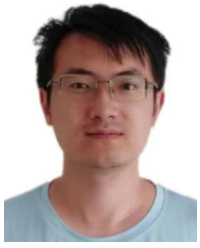
[30] P. Qiu et al., "Hijack vertical federated learning models with adversarial embedding," 2022, *arXiv:2212.00322*.

Ying He received the B.S. degree from Nanjing University in 2019, where she is currently pursuing the Ph.D. degree with the Department of Computer Science and Technology. Her current research interests include security and privacy in deep learning and federated learning.

Zhili Shen received the B.S. degree in software engineering from Chongqing University, Chongqing, China, in 2020, and the M.E. degree in computer technology from Nanjing University, Nanjing, China, in 2023. His current research interests include security and privacy.

Jingyu Hua (Member, IEEE) received the Ph.D. degree in computer science from Kyushu University, Japan, in 2012. Currently, he is a Professor with the Computer Science and Technology Department, Nanjing University, and also the Deputy Secretary of IEEE Computer Society Nanjing Chapter. His research interests include mobile security (e.g., smartphone security and wireless communication security), the privacy protection of big data, and network attack and defense.

Qixuan Dong received the B.S. degree in computer science and technology from Nanjing University, Nanjing, China, in 2022. His current research interests include security and privacy.



Jiacheng Niu received the B.S. degree in statistics and the M.E. degree in computer technology from Nanjing University, Nanjing, China, in 2019 and 2023, respectively. His current research interests include privacy and machine leaning.

Wei Tong (Member, IEEE) received the B.S., M.S., and Ph.D. degrees from Nanjing University in 2013, 2016, and 2019, respectively. He is currently an Assistant Researcher with the Department of Computer Science and Technology, Nanjing University. His research interests include data privacy and information security.

Xu Huang is currently a Senior Technical Engineer with Meituan. He is mainly responsible for the construction of Meituan's federated learning capabilities, system performance optimization, and project implementation. In the context of federated advertising, he has established links with multiple leading media outlets, significantly benefiting the business.

Chen Li is currently a Senior Technical Expert with Meituan. He is also the Head of the Development of the Machine Learning Platform System, responsible for the continuous iteration and delivery of algorithmic platform products.

Sheng Zhong (Senior Member, IEEE) received the B.S. and M.S. degrees in computer science from Nanjing University in 1996 and 1999, respectively, and the Ph.D. degree in computer science from Yale University in 2004. He is currently the Dean of the School of Software, Nanjing University. He has served on the editorial board of a number of journals. His research interests include security, privacy, and economic incentives.